

Mangofn formalization

Mangofn Development Team

August 2025

1 Preface

2 Introduction

3 Syntax of Mangofn

3.1 Reserved Words

The following are *reserved words* in Mangofn. These keywords are part of the language syntax and may not be used as identifiers.

Layout Constructs

- window
- column
- row
- border

UI Elements

- button
- text
- textbox
- checkbox
- radiobutton
- toggleswitch
- calendar

Boolean & Control Values

- true, false
- hidden, wrap

Styling Keywords

- fontfamily, fontsize, fontweight, fontstyle
- lineheight, textalign, textwrap, texttrim
- color, bgcolor, margin, width, height
- corner, density, fill, hug
- italic, underline, strikethrough
- center, left, right

Text Behavior

- word, character, notrim

Color Literals

- red, blue, green, yellow, pink, black, white

Language Constructs

- let, function

Symbols

- { }, (), :, ,, =

4 Lexical Grammar

Listing 1: Lexical rules

```
<ID> ::= [a-zA-Z] [a-zA-Z0-9_]* ;
<INT> ::= 0 | [1-9] [0-9]* ;
<STRINGLIT> : ''' (~["\\] | '\\\' .)* ''' ;
<BOOL> ::= 'true' | 'false' ;
<HEX> ::= '#' [0-9a-fA-F]{8} ;
```

4.1 Lexical Conventions

Mangofn source files are interpreted as sequences of ASCII characters. Whitespace and comments are ignored except where required to separate tokens. Identifiers and keywords are case-sensitive.

5 Syntactic Grammar

```
Prog          = Window [ "{" { UIElem } { Func } "}" ] ;
Window = "window" STRINGLIT [ INT INT [ STRINGLIT ] ] ;

UIElem        = "button" STRINGLIT [ "{" { Prop }+ "}" ]
               | "text" STRINGLIT [ "{" { Prop }+ "}" ]
               | "textbox" STRINGLIT
               | "checkbox" STRINGLIT
               | "radiobutton" STRINGLIT
               | "toggleswitch" STRINGLIT
               | "calendar"
               | "togglebutton"
               | "border" "{" { Prop }+ UIElem "}"
               | "column" "{" { Prop }+ { UIElem } "}"
               | "row" "{" { Prop }+ { UIElem } "}" ;

Prop          = PropName ":" PropValue
               | "onclick" ( ":" ID | "->" "{" { Stmt }+ "}" | "->" Stmt ) ;

PropName      = "bgcolor" | "wrap" | "corner" | "color" | "density" | "hidden"
               | "width" | "height" | "margin" | "id" | "fontfamily" | "fontsize"
               | "fontweight" | "fontstyle" | "lineheight" | "textalign" | "textwrap"
               | "texttrim" | "label" | "onclick" ;

PropValue     = Color | BOOL | Thickness | Size | STRINGLIT | INT
               | FontStyle | TextAlign | TextWrap | TextTrim ;

Func          = "function" ID "(" "{" { Stmt }+ "}" ;

Stmt          = "set" "(" PropName "," ID "," Exp ")"
               | "update" "(" ID "," "{" { Prop }+ "}" ")"
               | Exp ;

Exp           = INT | STRINGLIT | ID | BOOL | ID "(" ;

Thickness     = INT | INT "," INT | INT "," INT "," INT ;
Color         = "red" | "blue" | "green" | "yellow" | "pink"
               | "black" | "white" | HEX ;
```

```

Size          = INT | "fill" | "hug" ;
FontStyle     = "underline" | "italic" | "strikethrough" ;
TextAlign     = "center" | "left" | "right" ;
TextWrap      = "overflow" | "wrap" | "forcewrap" ;
TextTrim      = "word" | "character" | "notrim" ;

```

6 Semantics of Mangofn

6.1 Denotational Semantics

Start with the clean, compositional part:

```

[ button(label, props) ] : Env \rightarrow Node
[ column(props, elems) ] : Env \rightarrow Tree

```

Each syntactic form denotes a pure value in your domain of UI trees. Here you can play with

1. Fix-point definitions (recursive layouts)
2. Domain equations ($UI = elem \times Props \times [UI]$)
3. Functional mappings ($[Prog] : Env \rightarrow UIState$)

6.2 Constraint / Attribute Grammar Semantics - Layout

Model layout as attribute propagation or constraint solving.

```

attribute inherited availableWidth : Int
attribute synthesized layoutBoxes : [Rect]

```

```

rule Column:
  distribute(children, availableWidth)

```

Use an attribute grammar formalism or an explicit constraint system:

```

child2.top = child1.bottom + margin
width(child) \leq parent.width

```

This lets you experiment with:

1. Constraint satisfaction semantics
2. Fix-point evaluation of attribute grammars
3. Declarative layout solvers (Cassowary-style)

6.3 Operational Semantics - Dynamic behaviour

Model the runtime transitions for interaction and statements.

```
( UI, Env, click(id) ) \rightarrow ( UI', Env' )
```

or for command execution:

```
( set(prop, id, v), \sigma ) \rightarrow \sigma[id.prop \mapsto v]
```

Here you can explore:

1. Small-step vs. big-step semantics
2. Confluence and determinism proofs
3. Modeling reactivity as a labelled transition system (LTS)

6.4 Static Semantics - Type Rules

Specify rules like:

```
\gamma \multimap width : Int \lor {fill, hug}  
\gamma \multimap onclick : Function | Block
```

You can practice:

1. Typing derivations and inference
2. Rule-based validation of props
3. Developing a kind of system for style/value categories

6.5 Axiomatic Semantics - Reasoning About Programs

Once you have an operational semantics, you can add Hoare-style reasoning:

```
{ hasProp(id, color, old) } set(color, id, red) { hasProp(id, color, red) }
```

or temporal logic:

That gives you experience writing and proving correctness properties over UI behaviour.

6.6 Abstract Interpretation - Static Analysis Playground

Use this layer to practice static reasoning or optimization techniques: - Prove that all color values are valid hex codes. - Infer possible sizes of elements before layout. - Approximate event dependency graphs.

This connects formal semantics with program analysis.

A Examples

```
window "Mango UI Test" 600 400 "icon.png" {
  text "Welcome to Mango UI!" {
    fontsize: 40
    color: #ff00ff
  }
  column {
    margin: 0

    button "Click me" {
      width: 1000
    }
    button "Another button"
    checkbox "Enable advanced mode"
  }
  row {
    margin: 20

    text "Horizontal layout preview" {
      color: white
      bgcolor: red
    }
    radiobutton "Option A"
    text "Additional details" {
      color: yellow
      bgcolor: blue
    }
  }
}
```